

Shadow Properties in Entity Framework Core

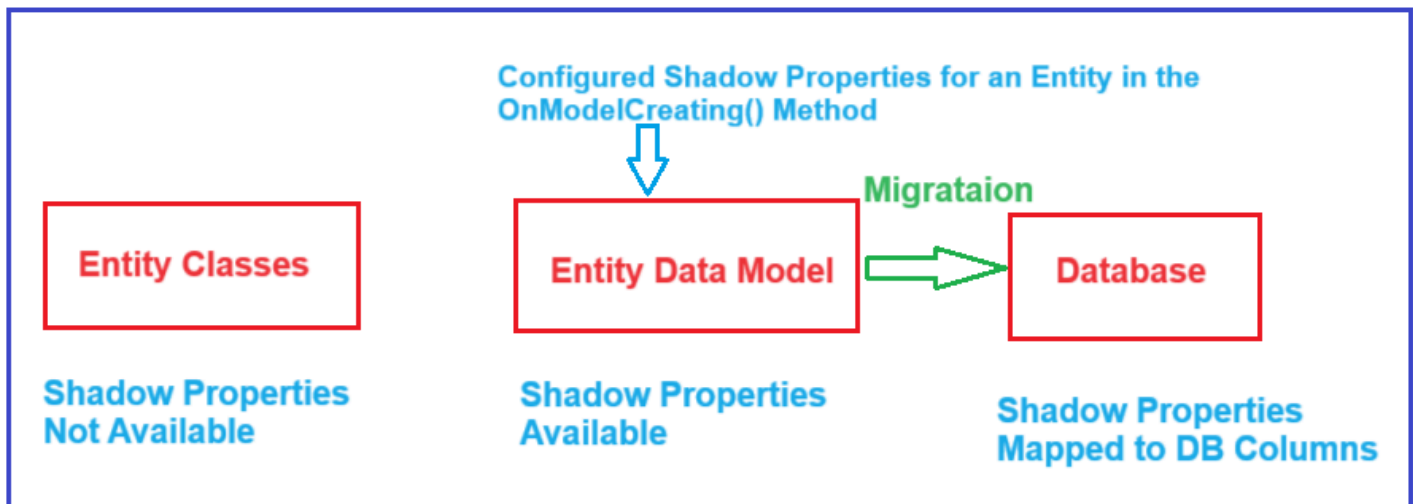
In this article, I will discuss **Shadow Properties in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Seed Data in Entity Framework Core](#) with Examples. Entity Framework Core introduced a new type of property called the “Shadow” Property, which did not exist in EF 6.x.

1. [What are Shadow Properties in EF Core?](#)
2. [Example to Understand Shadow Properties in Entity Framework Core](#)
3. [How Do We Check the Shadow Properties of an Entity in EF Core?](#)
4. [Creating Foreign Key Properties as Shadow Properties in EF Core](#)
5. [Shadow Property with Default Value in Entity Framework Core](#)
6. [When to use Shadow Properties in Entity Framework Core?](#)

What are Shadow Properties in EF Core?

Shadow properties in Entity Framework Core are fields not defined in your .NET entity class but are defined in the model and, hence, are part of the database schema. These properties can be used to store and retrieve values with the database without declaring them explicitly in your class model. Shadow properties are useful when working with database columns that don't have corresponding properties in our entity class.

This can be useful for fields that should be in the database but are not needed in the model class, like audit information like who created or modified a row and when these operations occurred. In our entities, we usually use fields like CreatedOn, LastModifiedOn, CreatedBy, LastModifiedBy, etc., to store audit information. Shadow Properties are configured in the OnModelCreating() method of the context class. For a better understanding, please have a look at the following diagram:



As you can see in the above diagram, the Shadow Properties in EF Core are not part of our entity classes. So, we cannot access these Shadow Properties as we access the other properties of an entity. Shadow Properties can only be configured for an entity type while building an Entity Data Model and mapped to a database column.

Example to Understand Shadow Properties in Entity Framework Core:

Shadow properties can be used for various purposes, such as:

- **Auditing:** Storing information like creation and modification timestamps.
- **Concurrency Control:** Managing concurrency with fields like timestamps or version numbers.
- **Tracking Additional information:** Storing metadata or flags related to entities.

Consider a real-time example of using shadow properties in an ASP.NET Core application with Entity Framework Core. Imagine you have a blog post entity that wants to track when each post was created and last updated, but you don't want these audit fields to be part of your domain model. Let us see how we can implement this using shadow Properties.

Define the BlogPost Entity Without Audit Properties:

Create a class file named **BlogPost.cs** and copy and paste the following code. Here, we have not added the CreatedAt or LastUpdatedAt properties.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class BlogPost
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public string Content { get; set; }
        // No CreatedAt or LastUpdatedAt properties here
    }
}
```

Configure Shadow Properties in the OnModelCreating Method of your DbContext:

Shadow properties are typically defined in the OnModelCreating method of your DbContext class using the Fluent API. So, to define a Shadow Property in EF Core, we can use the modelBuilder API in our DbContext class's OnModelCreating method. You can use the Property method to configure a shadow property. In our example, we have created two Shadow Properties named **CreatedAt** and **LastUpdatedAt** for the **BlogPost** class. So, modify the context class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<BlogPost>().Property<DateTime>("CreatedAt");
            modelBuilder.Entity<BlogPost>().Property<DateTime>("LastUpdatedAt");
        }
        public DbSet<BlogPost> BlogPosts { get; set; }
    }
}
```

Set the Shadow Properties Values when Saving Changes:

We can do the same by overriding our context class's **SaveChanges()** method, or while defining the Shadow Property, we can provide the default value for the Shadow Property. Let us first see how we can override the **SaveChanges()** method to set the Shadow Properties Values in EF Core.

For a better understanding, please have a look at the following image. As you can see, we have overridden the **SaveChanges()** method. Within the **SaveChanges()** method, we loop through all the Entities whose Entity Type is **BlogPost** and check whether the entity state is added or modified. If the Entity is in Added State, we set both **CreatedAt** and **LastUpdatedAt** Shadow Properties values to the current date. If the Entity is in the Modified State, we set the **LastUpdatedAt** Shadow Properties value to the current date.

```
public override int SaveChanges()
{
    var timestamp = DateTime.UtcNow;
    foreach (var entry in ChangeTracker.Entries<BlogPost>())
    {
        if (entry.State == EntityState.Added)
        {
            entry.Property("CreatedAt").CurrentValue = timestamp;
            entry.Property("LastUpdatedAt").CurrentValue = timestamp;
        }
        else if (entry.State == EntityState.Modified)
        {
            entry.Property("LastUpdatedAt").CurrentValue = timestamp;
        }
    }

    return base.SaveChanges();
}
```

So, modify the context class as follows:

```
using Microsoft.EntityFrameworkCore;
using System.Diagnostics;
using System.Threading.Channels;
using System;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<BlogPost>().Property<DateTime>("CreatedAt");
        }
    }
}
```

```

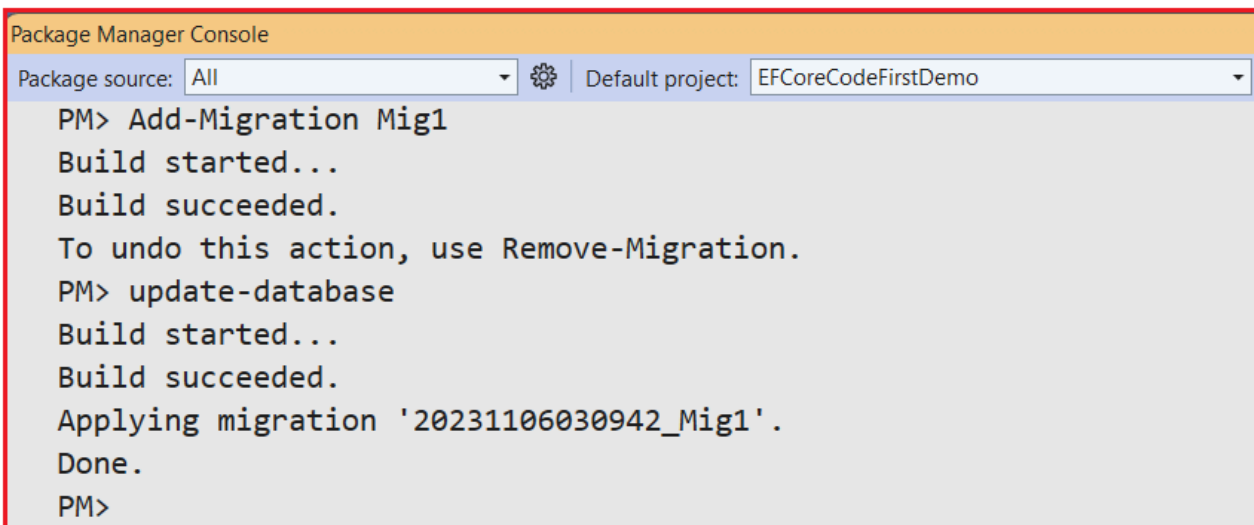
        modelBuilder.Entity<BlogPost>().Property<DateTime>("LastUpdatedAt"
    }

    public override int SaveChanges()
    {
        var timestamp = DateTime.UtcNow;
        foreach (var entry in ChangeTracker.Entries<BlogPost>())
        {
            if (entry.State == EntityState.Added)
            {
                entry.Property("CreatedAt").CurrentValue = timestamp;
                entry.Property("LastUpdatedAt").CurrentValue = timestamp;
            }
            else if (entry.State == EntityState.Modified)
            {
                entry.Property("LastUpdatedAt").CurrentValue = timestamp;
            }
        }
        return base.SaveChanges();
    }
    public DbSet<BlogPost> BlogPosts { get; set; }
}
}

```

Generate Migration and Apply Database Changes:

Open the Package Manager Console and Execute the **add-migration** and **update-database** commands as follows. You can give any name to your migration. Here, I am giving **Mig1**. The name that you are giving it should not be given earlier.

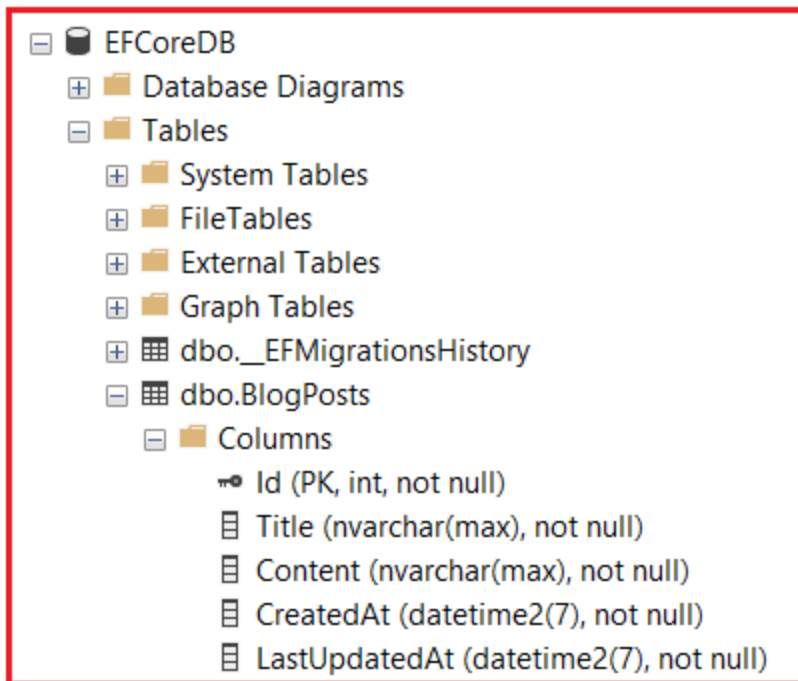


```

Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231106030942_Mig1'.
Done.
PM>

```

Now, verify the database, and you should see the Shadow Properties in the table as shown in the below image:



Save BlogPosts with Shadow Properties:

When you add or update a BlogPost, Entity Framework Core will automatically handle the shadow properties for us. Entity Framework Core will update the "CreatedAt" and "UpdatedAt" shadow property values in the database. For a better understanding, please modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                var blogPost = new BlogPost
                {
                    Title = "EF Core",
                    Content = "IT is an ORM Framework"
                };
                using var context = new EFCoreDbContext();
                context.BlogPosts.Add(blogPost);
                context.SaveChanges();
                Console.WriteLine("New BlogPost Added..");
                // Entity Framework Core will set the "CreatedAt" and
                "LastUpdatedAt" Shadow Properties Value
                blogPost.Content = "Entity Framework Core is Updated";
                context.SaveChanges();
                // Entity Framework Core will update the "LastUpdatedAt"
                shadow property value.
                Console.WriteLine("BlogPost Updated..");
                Console.Read();
            }
        }
    }
}
```



```

        })
        .ToList();
    foreach (var blogPost in blogPostsWithAudit)
    {
        Console.WriteLine($"Title: {blogPost.Title}, CreatedAt:
{blogPost.CreatedAt}, LastUpdatedAt:{blogPost.LastUpdatedAt}");
        Console.WriteLine($"\\tContent: {blogPost.Content}");
    }
    Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
}
}
}

```

Output:

```

Title: EF Core, CreatedAt: 08-11-2023 11:16:20, LastUpdatedAt:08-11-2023 11:16:21
Content: Entity Framework Core is Updated

```

Note: In our example, the BlogPost class doesn't contain CreatedAt or LastUpdatedAt properties, but they are part of the BlogPosts table in the database. When we add or update a BlogPost, the SaveChanges method sets these shadow properties automatically. When querying, we can still retrieve these values even though they are not part of the BlogPost class using the **EF.Property** method.

How Do We Check the Shadow Properties of an Entity in EF Core?

In Entity Framework Core, we can inspect an entity's shadow properties at runtime using the DbContext and its associated ChangeTracker. Here's how you can check for shadow properties:

- **Retrieve the Entry for the Entity:** Use the DbContext.Entry method to get the EntityEntry for the entity you're interested in.
- **Use the Metadata Property:** The EntityEntry has a Metadata property that provides access to metadata about the entity, including information on shadow properties.
- **Iterate Over Properties:** You can then iterate over the Metadata.GetProperties() collection and check for properties that are shadow properties using the IsShadowProperty flag.

Here's an example of how to write a method that prints out the names of all shadow properties for a given entity:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {

```

```

var blogPost = new BlogPost
{
    Title = "EF Core",
    Content = "IT is an ORM Framework"
};
using var context = new EFCoreDbContext();
context.BlogPosts.Add(blogPost);
context.SaveChanges();
// Assuming you have a DbContext instance named context
PrintShadowProperties(context, blogPost);
Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
}
public static void PrintShadowProperties<TEntity>(DbContext context,
TEntity entity) where TEntity : class
{
    var entry = context.Entry(entity);
    var shadowProperties = entry.Metadata.GetProperties()
        .Where(p =>
p.IsShadowProperty())
        .Select(p => p.Name);
    Console.WriteLine($"Shadow Properties for
{typeof(TEntity).Name}:");
    foreach (var propName in shadowProperties)
    {
        Console.WriteLine(propName);
    }
}
}
}

```

Output:

```

Shadow Properties for BlogPost:
CreatedAt
LastUpdatedAt

```

Creating Foreign Key Properties as Shadow Properties in EF Core:

Let's create an example where we have two entities, Blog and Post, and we will configure a shadow property to act as a foreign key from Post to Blog without including the foreign key property in the Post class. Here's how you could set up the foreign key as a shadow property in Entity Framework Core:

Define the Blog and Post entities without foreign key properties:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Blog
    {
        public int BlogId { get; set; }
        public string Url { get; set; }
        public List<Post> Posts { get; set; }
    }
    public class Post
    {
        public int PostId { get; set; }
        public string Title { get; set; }
        public string Content { get; set; }
        // No BlogId foreign key property here
        public Blog Blog { get; set; }
    }
}

```

Configure the shadow foreign key property in the OnModelCreating method:

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Post>()
                .HasOne(p => p.Blog)
                .WithMany(b => b.Posts)
                .HasForeignKey("BlogId"); // Configure BlogId as a shadow foreign key
        }

        public DbSet<Blog> Blogs { get; set; }
        public DbSet<Post> Posts { get; set; }
    }
}

```

Setting the shadow foreign key value when adding a new Post:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {

```

```

{
    static async Task Main(string[] args)
    {
        try
        {
            using var context = new EFCoreDbContext();
            var blog = new Blog { Url = "http://dotnettutorials.net" };
            context.Blogs.Add(blog);
            context.SaveChanges();
            var post = new Post { Title = "Hello World", Content =
"Welcome to my Blog!" };
            context.Posts.Add(post);
            // Set the shadow foreign key value
            context.Entry(post).Property("BlogId").CurrentValue =
blog.BlogId;
            context.SaveChanges();
            Console.Read();
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }
    }
}

```

Using the shadow foreign key property in a query:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                using var context = new EFCoreDbContext();
                var postsWithBlogs = context.Posts
                    .Select(p => new
                    {
                        Post = p,
                        BlogId = EF.Property<int>(p, "BlogId") //
Access the shadow property
                    })
                    .ToList();
                Console.Read();
            }
            catch (Exception ex)

```

```

        {
            Console.WriteLine($"Error: {ex.Message}");
        }
    }
}

```

In this example, the Post class does not contain a BlogId property but is part of the Posts table in the database as a shadow property. This allows us to have a clean domain model without the foreign key property while still being able to set and retrieve the foreign key value using EF Core functionalities.

Shadow Property with Default Value in EF Core

In Entity Framework Core, we can configure shadow properties with a default value using the Fluent API. This can be useful for columns that should always start with a specific value when a new record is created, such as a default state, creation date, or a flag. Here's an example of how to set up a shadow property with a default value:

Define your entity without the property you want as a shadow property:

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class User
    {
        public int UserId { get; set; }
        public string Name { get; set; }
        // IsActive is not included here, it will be a shadow property with a
        default value
    }
}

```

Configure the shadow property and its default value in the OnModelCreating method:

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<User>().Property<bool>("IsActive").HasDefaultValue(true);
        }
        public DbSet<User> Users { get; set; }
    }
}

```



```

        IsActive = EF.Property<bool>(u, "IsActive")
// Access the shadow property
    })
    .ToList();
    Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
}
}
}
}

```

This LINQ query includes the shadow property `IsActive` in the result set using the **EF.Property<T>** method. This allows you to read the value of the shadow property even though it's not part of your entity class.

When to use Shadow Properties in Entity Framework Core?

Shadow properties in Entity Framework Core are useful in various scenarios:

- **Auditing:** Shadow properties are often used for audit data, like when an entity was created or last modified. This keeps audit details out from the domain model but still in the database.
- **Querying:** You can query shadow properties using LINQ, but you must use the **EF.Property** static method to refer to them in your queries.
- **Migrations:** When you add or change shadow properties, you must create a new migration to update the database schema.
- **Data Models and Mappings:** Note that shadow properties are part of the EF model, but they won't be visible in your C# data models. This can sometimes make it harder to understand the complete data model just by looking at the classes.
- **Model Clarity:** While useful, overuse of shadow properties can lead to a less transparent model, where significant parts of the data model are not immediately visible in the code.
- **Database Provider Compatibility:** Ensure that any default values or behaviors you set for shadow properties are compatible with your database provider.
- **Modeling Database Concerns:** Sometimes, there are columns in a database table that have no direct relevance to the domain model's behavior or business logic but are necessary for database operations or constraints. These can be handled as shadow properties.
- **Encapsulation:** When you want to keep certain data private to the entity and not expose it to the domain directly, shadow properties can be useful. They allow the data to be stored and retrieved without defining it in the domain model.
- **Soft Delete:** A shadow property can hold this flag in scenarios where you do not delete records but mark them as deleted with a flag.

In the next article, I will discuss [Global Query Filters in Entity Framework Core](#) with Examples. In this article, I try to explain **Shadow Properties in Entity Framework Core (EF Core)** with Examples. I hope you enjoy this Shadow Properties in EF Core article.